Plano do TCC: biblioteca prossumidor

O que é, o que já foi feito e o que falta | João Carlos Jachenski Junior | Lactec | revisão de 09/10/2026

> **Em uma frase:** o TCC é uma biblioteca em Python que prevê quanta energia uma casa ou usina solar vai **gerar** e **consumir** no dia seguinte, desde a leitura do medidor até um modelo pequeno que roda num aparelho barato (Raspberry Pi ou ESP32).

1. A proposta

Hoje o projeto tem dezenas de scripts soltos que repetem as mesmas contas (datas de teste, métricas, limpeza de dados). Quando uma coisa muda, é preciso corrigir à mão em muitos lugares, e já houve erros por isso. A ideia é juntar tudo numa **biblioteca única, testada e reaproveitável**. Ela não reinventa o que já existe: usa bibliotecas prontas (pvlib, LightGBM, modelos de IA públicos) e só acrescenta o que é próprio deste projeto.

Parte da biblioteca	O que faz, em poucas palavras
io	Lê os arquivos reais: inversor Fronius, medidores das casas, dados de clima.
qualidade	Acha dados ruins: buracos, noite, troca de horário, casa sem painel solar.
fisica	Calcula a posição do sol e quanta luz chegaria num dia sem nuvens.
protocolo	Regras únicas de teste: que dias prever, como comparar, como medir o erro.
previsao	Um jeito único de usar qualquer modelo, do mais simples aos modelos de IA.
diagnostico	Descobre painel sujo ou com defeito usando só os dados do inversor.
embarcado	Encolhe o modelo para caber e rodar num aparelho pequeno.
economia	Calcula a conta de luz: tarifas da ANEEL, bandeiras e a Lei 14.300.

2. O que já foi feito

- **Teste grande de modelos de previsão. 51 modelos** em 6 testes para o artigo (usinas, geração FV das casas e consumo das casas, cada um sem e com o clima do dia) e mais o regime com a previsão do tempo real nas usinas: 243 resultados. Cada modelo é retreinado a cada período de teste, e as diferenças são checadas com testes estatísticos. Entraram também os clássicos (ARIMA, ETS, Theta, MSTL, Ridge, Random Forest, MLP, k-NN).
- **As 20 casas foram identificadas** (Dingle, Irlanda, domínio público) e ganharam o clima medido da estação local. O relógio delas (hora local com horário de verão) foi descoberto comparando a geração solar com a posição do sol.
- **Uso acadêmico liberado** (05/10/2026) – etapa E0.
- **Base da biblioteca** (etapa E1): pacote instalável, checagem dos dados de entrada (com horário de verão), dados de exemplo, **304 testes automáticos** no GitHub, incluindo um que recalcula os 243 resultados publicados e exige que batam exatamente.
- **Revisão de 09/10/2026:** erros de método achados e corrigidos (código da casa tratado como número em três modelos; planilhas geradas no meio de uma rodada), e **testes-piloto** que decidiram dois comportamentos da biblioteca (seção 4).

3. Os dados

Dados	Origem	Pode publicar?
2 usinas solares	Cascavel (3,96 kWp) e Curitiba (∼9,6 kWp), vindas do professor responsável pelos dados.	**Com a permissão dele** (que está aberto). Até a permissão explícita, ficam privadas e com os nomes trocados pela cidade.
20 casas	Comunidade de energia em Dingle, Irlanda, 2020, medição a cada minuto, com estação meteorológica. Khadem et al. (2024), *Scientific Data*, doi:10.1038/s41597-024-03454-2.	**Sim.** Domínio público (CC0). Basta citar o artigo.

A Lactec só entra se o projeto for ligado formalmente a um projeto dela. Fora disso, as decisões de licença e publicação são do autor, com a permissão do professor para os dados das usinas.

4. O que os testes ensinaram para a biblioteca

- **O clima pesa cerca de 10 vezes mais que o modelo.** Nas usinas, dar o clima ao modelo sobe o skill em +0,46 (mediana); trocar o 1º pelo 5º modelo muda 0,04. Por isso o maior investimento vai para **obter clima e previsão do tempo melhores**.
- **A mediana de todos os modelos empata com o melhor nos 6 testes.** Vira a saída padrão. *Testado:* escolher o melhor da rodada anterior piora em todos os casos (até $-0{,}23$), e ficar só com os 5 ou 10 melhores não é estável. A biblioteca **não** escolhe modelo.
- **Correção de viés só no consumo.** *Testado:* os modelos de IA preveem $\sim$20% a menos de consumo; corrigir pelo viés dos últimos 14 dias leva o erro médio a $<$1% e sobe a mediana de +0,191 para +0,207 (empata com o melhor). Na geração solar a mesma correção **piora** até $-0{,}10$: lá ela fica desligada.
- **O consumo de uma casa, hora a hora, é em grande parte imprevisível** (R^2 0,37); somando as 20 casas sobe para 0,64, e o total do dia para 0,58. A biblioteca mede o erro em várias resoluções (hora, blocos de tarifa, pico da noite, dia), para cada uso escolher a sua.
- **Medir, não supor.** O fuso das casas estava errado e só apareceu comparando a geração com o sol. Isso vira uma verificação automática do relógio no módulo de qualidade.
- **Cada modelo declara o que exige**: GPU, licença dos pesos e se envia dados para fora (um deles roda em servidor pago; dados privados nunca vão para lá sem permissão).

5. Para quem é a biblioteca (decidido em 09/10/2026)

> **Internacional no núcleo, com uma camada brasileira.** Código e documentação principal em inglês, para o setor elétrico de qualquer país e para a comunidade de pesquisa; módulo e guia em português para o setor brasileiro. Mantém aberta a opção de mestrado ou doutorado no exterior.

Público	O que usa	O que a biblioteca entrega
Setor elétrico brasileiro	previsão para usinas e prossumidores, conta de luz, tarifa branca, Lei 14.300	módulo de economia brasileiro (ANEEL, bandeiras, Copel), guia em português
Setor elétrico mundial	previsão de geração e consumo, baterias, tarifas por horário	núcleo em inglês; tarifas por horário genéricas; fontes de clima de qualquer lugar
Pesquisa e desenvolvimento	comparar modelos de forma justa, reproduzir resultados	protocolo, benchmark aberto (casas de Dingle), adaptadores, artigo de software (JOSS ou SoftwareX)

O nome passa a ser em inglês (`prosumer` está livre no PyPI em 09/10/2026; conferir de novo na hora). A troca de nomes no código entra no começo da E2, antes de o núcleo crescer.

6. O plano: testar antes de investir

Toda melhoria começa por um **piloto curto** (1 a 4 horas, num caso só). Só vira trabalho completo se passar no critério da coluna "Portão". E nada é descartado sem evidência: modelo só sai depois de estudado e testado (seção 7). Horas com cerca de 15 horas por semana.

Etapa	O que é	Portão (piloto)	Horas
F0 – Decisões	Licença (do autor); permissão do professor para os dados das usinas; comprar Raspberry Pi 5 e ESP32.	–	0
F1 – Fechar o teste	Congelar os ambientes; avaliação por resolução (hora, tarifa, pico, dia); correção de viés do consumo (já testada); ARIMA e MLP; TimesFM-3, FlowState e BuildingsBench.	ARIMA: 1 caso em menos de 2 h; MLP: ganho $\geq$0,05; modelos: entrar no grupo da frente num caso.	17
A – Modelos pendentes	Estudar cada um dos 13 (como funciona, a matemática, o que pode acrescentar, como fazer rodar) e testar num caso (seção 7).	sai só com evidência; entra no teste completo se empatar com o melhor ou trouxer algo que nenhum outro traz	40
F2 – Pilotos de melhoria	(a) combinar 2–3 previsões do tempo; (b) estimar a inclinação e a direção de cada telhado pelos dados; (c) prever uma faixa (quantis); (d) atualizar a previsão ao meio-dia; (e) ajuste fino de um modelo de IA nos nossos dados; (f) modelo direto para o total do dia.	cada um: skill $+$0,02 ou mais num caso, ou faixa bem calibrada (c).	22
F2b – Levar para a biblioteca	Só os pilotos que passaram viram funções da biblioteca.	estimativa	40
E2 – Versão 0.1	Troca dos nomes para inglês; núcleo: ler e limpar dados (com o relógio verificado pelo sol), física do sol, regras de teste em várias resoluções, ranking e empate, modelos com o que exigem declarado, mediana como saída, dados reais de exemplo (Dingle).	testes automáticos; cobertura $\geq$80%	130
E3 – Checagem 1	Refazer o teste grande usando só a biblioteca.	os 243 resultados batem	30
E4 – Versão 0.2	Diagnóstico de painel sujo ou com defeito; modelos pequenos para aparelhos (TTM, SparseTSF, Ridge, Random Forest e o que a etapa A indicar).	diagnóstico de Cascavel reproduzido	130
E5 – Checagem 2	Medir tamanho, acerto, velocidade e energia no Raspberry Pi 5 e no ESP32.	tabela medida no aparelho	50
E6 – Versão 0.3	Economia: tarifas por horário genéricas e o módulo brasileiro (tarifa branca, bandeiras, Lei 14.300); documentação em inglês, guia em português; publicação.	licença escolhida	110
E7 – Escrita	O TCC. Começa já, em paralelo.	aprovação do orientador	80
Total			**649**

Total de $\sim$43 semanas (10 meses). Subprodutos, sem atrasar a biblioteca: artigo de congresso com os regimes A e B ($\sim$25 h; a planilha já está pronta) e artigo de software ($\sim$30 h).

7. Modelos pendentes: estudar e testar antes de decidir

Nenhum destes foi descartado. Cada um tem uma razão para poder acrescentar algo, e só sai se o estudo e o piloto mostrarem que não acrescenta. “Viabilidade” diz o que falta para ele rodar.

Modelo	Como funciona	O que pode acrescentar	Viabilidade
Moirai 1	Transformer de máscara com atenção “any-variate”: várias séries e variáveis numa sequência só, com saída em mistura de distribuições.	Clima como entrada sem treino (o Moirai 2 testado só rodou sem clima).	Ambiente moirai já existe.
CycleNet	Aprende um ciclo diário/semanal explícito e um modelo linear prevê só o resto.	Muito pequeno: candidato ao embarcado (mesmo autor do SparseTSF, que empatou no consumo).	Precisa de adaptador.
VisionTS	Corta a série pelo período (24 h) em uma imagem e usa um autoencoder de imagens pré-treinado para “completar” o futuro.	A sazonalidade diária forte da FV é o caso ideal dele.	Pacote pronto.
MOMENT	Codificador T5 pré-treinado reconstruindo pedaços mascarados; prever exige treinar uma cabeça linear.	Testa “modelo grande congelado + cabeça pequena”: ajuste fino barato.	Tem versão pequena.
Lag-Llama	Transformer decodificador que recebe valores defasados (ontem, semana passada) e devolve uma distribuição.	Previsão em faixa sem treino (piloto F2c).	Código no GitHub.
DUET	Agrupa períodos parecidos (mistura de especialistas) e agrupa séries que se influenciam.	Usa a correlação entre as 20 casas da mesma comunidade.	Adaptador (TFB).
Crossformer	Atenção em duas etapas: no tempo e entre séries.	Idem: dependência entre casas e entre geração e clima.	Adaptador (TSLib).
Mamba, S-Mamba	Espaço de estados seletivo: custo linear no tamanho do histórico.	Inferência barata com histórico longo: interessa ao embarcado.	Versões em PyTorch puro, sem os kernels CUDA.
xLSTM	LSTM com portas exponenciais e memória em matriz.	O melhor modelo (TiRex) é baseado nele: falta a versão treinada nos nossos dados.	Linux/WSL com compilador.
TimeGrad	Rede recorrente + difusão a cada passo: amostra futuros possíveis.	Previsão em faixa.	PyTorchTS; liga ao piloto F2c.
Time-LLM, LLMTime	Usam um modelo de linguagem: reprogramam a série para o espaço do texto (Time-LLM) ou escrevem os números como texto (LLMTime).	Medir nos nossos dados se o LLM ajuda (a literatura diz que não).	Base menor (GPT-2) ou mais memória.
TimeGPT	Modelo fundacional comercial, por API, com variáveis externas.	Referência comercial.	Pago; envia os dados para fora: só as casas (CC0).

Descartado com evidência (já testado): escolher automaticamente “o melhor modelo” (piora ou é instável) e corrigir o viés da geração solar (piora até $-0{,}10$). O TimesNet ficou só no regime A porque **ignora** o clima (previsão idêntica com o clima embaralhado).

8. Cronograma sugerido

Etapa	out	nov	dez	jan	fev	mar	abr	mai	jun	jul	ago
F0 Decisões	■										
F1 Fechar o teste	■										
A Modelos pendentes		■	■								
F2 Pilotos	■	■									
E2 Versão 0.1		■	■	■							
E3 Checagem 1					■						
E4 Versão 0.2					■	■	■				
E5 Hardware real								■			
E6 Versão 0.3									■	■	
E7 Escrita do TCC	■	■	■	■	■	■	■	■	■	■	■
Artigos (subprodutos)		■	■								■

Outubro de 2026 a agosto de 2027. A etapa A roda em paralelo com a E2, na máquina de casa (GPU). Os artigos (cinza) são subprodutos: o de congresso aproveita a planilha pronta; o de software, depois da publicação.

9. O que pode atrasar (resolver logo)

- **Permissão do professor** para publicar os dados das usinas, por escrito (e-mail basta).
- **Licença da biblioteca**: escolha do autor; definir antes da E6.
- **Comprar o Raspberry Pi 5 e o ESP32** com antecedência (etapa E5).
- **Memória da máquina de casa**: três rodadas morreram por falta de RAM. Rodar em lotes pequenos e não misturar filas de CPU e GPU.